Claude Code'u iyi kullanmanın kontrol listesi

19 bölüm, 97 madde. İlk sayfada en önemli on madde, her maddede neden, nasıl, kopyalanabilir örnek ve doğrulama.

Claude Code 2.1.284 · Ekim 2026
Beş aylık gerçek kullanım verisinden ve Claude Code belgelerinden (code.claude.com) derlendi.

Nasıl okunur

"İyi kullanmak" burada hız ya da çok özellik kullanmak demek değil. Ölçüt üç tane: aynı işi daha az mesajla bitirmek, aynı limitle daha çok iş çıkarmak ve sonucu kanıtla teslim almak. Bilgisayar başında değilken de ayakta kalan bir düzen bu üçünün toplamı.

Her bölüm kısa bir gerekçeyle başlıyor, maddeler getirisi yüksekten düşüğe sıralı. **Neden** satırı önemini, **nasıl** satırı komutu ya da ayarı söyler. Örneği olan maddelerde kopyalanabilir bir parça var. **Doğrula** satırı işin gerçekten olduğunu neye bakarak anlayacağını gösterir. Doğrulanamayan madde yapılmış sayılmaz.

Temel etiketli 25 madde her kullanıcı için, ilk haftada yapılabilir. **İleri** etiketli 72 madde otomasyon, gözetimsiz koşum, ekip ve çok oturumla çalışanlar için.

Belge Claude Code'u (CLI ve masaüstü uygulaması) kapsıyor. API ile uygulama geliştirme ve ücretli araç önerisi kapsam dışında. Özellik adları 16 Ekim 2026'da code.claude.com belgeleriyle doğrulandı. Bir komut ya da ayar çalışmıyorsa sürümünü `claude --version` ile kontrol et ve güncel belgeye bak.

Gerçek bir kullanımdan sayılar

Listenin önceliklerini belirleyen veri: tek bir kullanıcının beş aylık oturum kayıtları. Senin dağılımın farklı olabilir. Bu sayılar sıralamanın gerekçesi, senin için hedef değil.

425 oturum, 2.372 mesaj

Mayıs-Ekim 2026, yaklaşık 40 proje.

104 kez "devam"

Mesajların yaklaşık %7'si yalnız işi ileri itmek için. Bitiş ölçütü verilmeyen işlerin izi.

55 kez bağlam doldu

Oturum özetle devam etti, 35 kez elle sıkıştırıldı, 6 oturum 100 mesajı geçti.

1.073 alt ajan çağrısı

Neredeyse hepsi en güçlü modelde koştu. Varsayılanı değiştirmek tek satır.

3.758 simülatör aracı çağrısı

Elle tıklayarak doğrulama, otomatik teste çevrilebilecek en büyük kalem.

Sistem yükü 240-475

Aynı anda çok oturumun açık olduğu bir akşam. Testlerin rastgele ölmesinin sebebi.

Önce bu on madde

Bütün listeyi okumaya vaktin yoksa buradan başla. En çok getirisi olanlar, çoğu birkaç dakikalık iş.

1 **Bitiş ölçütünü baştan yaz** madde 3.1
Ölçüt yoksa iş parça parça ilerler ve her parçada seni bekler.

2 **Bir özellik için bir oturum, sonunda devir dosyası** madde 2.1
Uzun oturum hem limit yer hem de sıkıştırmada kararları kaybettirir.

3 **Her projede 100 satırı geçmeyen bir CLAUDE.md** madde 1.2
Derleme komutu ve tuzaklar her oturumda yeniden keşfedilmesin.

4 **Projeye izin listesi** madde 6.1
Gözetimsiz koşumu ayakta tutan en güvenilir şey projeye ait liste.

5 **Gizli dosyaları okuma dışı bırak** madde 7.1
Anahtarın bağlama girmesi, sonra bir çıktıya ya da commit'e sızması demek.

6 **Alt ajanları Sonnet'e al** madde 5.1
Okuma, tarama, doğrulama işlerinde en güçlü model gerekmez. Ana oturum yine güçlü modelde kalır.

7 **"Geçti" yerine sayıyı oku** madde 12.1
Eşleşmeyen bir test filtresi sıfır test koşup başarılı diyebilir.

8 **Her proje git'te, commit Claude'da** madde 15.1
Küçük commit küçük geri alma demek. `.bak` kopyaları geri alma değil, karmaşa üretir.

9 **Düşünme bütçesini işe göre ayarla** madde 5.3
Mimari karar ile dosya yeniden adlandırma aynı bütçeyi istemez. "Think hard" gibi ifadeler sıradan metin olarak geçer, hiçbir ayarı değiştirmez. Yalnız `ultrathink` o tur için ek bir talimat ekler, effort ayarı yine aynı kalır.

10 **Eleştiriyi yazandan ayrı bağlama ver** madde 9.1
Bir şeyi yazan model, onu gözden geçirirken onaylamaya meyillidir.

1 Talimat katmanları

Claude her oturuma sıfırdan başlar. Seni ve projeni, talimat dosyalarına yazdığın kadar tanır. Dosya kısa ve isabetliyse her mesajda işe yarar. Uzunsa her mesajda yer kaplar ve kurallar birbirini sulandırır.

- [ ] **1.1 Global CLAUDE.md'ye yalnız varsayılan davranışı değiştiren kuralları yaz.** TEMEL

  NEDEN Dosya her mesajda bağlama girer. Uzadıkça tek bir kuralın ağırlığı düşer.

  NASIL `~/.claude/CLAUDE.md` . Genel tavsiye değil, davranış yaz.

  ```
  # Kötü: genel tavsiye
  - Net ve dikkatli ol.

  # İyi: davranışı değiştiriyor
  - Commit'i sormadan at, push'u sormadan atma.
  - Seçenekleri harflendir, hangisini neden önerdiğini yaz.
  ```

  DOĞRULA Her kural için sor: silsem davranış değişir mi? Değişmiyorsa sil.

- [ ] **1.2 Her aktif projede 100 satırı geçmeyen bir CLAUDE.md tut.** TEMEL

  NEDEN Derleme komutu ve tuzaklar her oturumda yeniden keşfedilmesin.

  NASIL `/init` ile başla, sonra buda. Komutları birebir yaz.

  ```
  # Proje: fiyat-izleyici
  ## Komutlar
  - Test: npm test -- --run
  - Tek dosya: npm test -- src/parser.test.ts
  ## Tuzaklar
  - data/ altındaki JSON elle düzenlenmez, scripts/topla.mjs üretir.
  - Saat dilimi her yerde Europe/Istanbul.
  ```

  DOĞRULA Yeni oturumda dosya okutmadan "test komutu ne?" diye sor. Doğru cevap gelmeli.

- [ ] **1.3 Makineye özgü bilgiyi `CLAUDE.local.md`'ye yaz ve gitignore'la.** İLERİ

  NEDEN Repo başka makinede ya da bulut oturumunda açılınca yanlış yol talimatı kalmaz.

  NASIL Proje kökünde `CLAUDE.local.md` : venv yeri, yerel port, cihaz adı.

  DOĞRULA `git check-ignore CLAUDE.local.md` eşleşme döndürmeli.

- [ ] **1.4 Yalnız belli dosyalarda geçerli kuralı `.claude/rules/` altına koy.** İLERİ

  NEDEN Kural yalnız eşleşen dosyaya dokunulunca yüklenir, geri kalan zamanda bağlamı doldurmaz.

  NASIL Frontmatter'da `paths` . Kullanıcı düzeyi için `~/.claude/rules/` .

  ```
  ---
  paths: ["**/*.swift"]
  ---
  - Görünüm dosyalarında iş mantığı yok, ViewModel'e taşı.
  - Yeni ekran eklenince Previews da eklenir.
  ```

  DOĞRULA Eşleşen bir dosyayı okut, ardından `/context` ile kuralın yüklendiğini gör.

- [ ] **1.5 Proje dışındaki bir dosyayı her oturuma sokmak için `@~` import'una güvenme.** İLERİ

  NEDEN Proje dışı import proje başına onay istiyor. Onaysız projede, worktree'de ve headless koşumda sessizce yüklenmeyebiliyor.

  NASIL Kesin yüklenmesi gerekiyorsa SessionStart hook'u kullan (Hook'lar bölümü).

  DOĞRULA `claude -p` ile, okuma araçları kapalıyken dosyadaki bir ayrıntıyı sor.

- [ ] **1.6 Başka bir ajanla paylaşılan repoda kuralları tek dosyada tut.** İLERİ

  NEDEN İki dosya ayrı ayrı güncellenince birbirinden kopar.

  NASIL Kurallar `AGENTS.md`'de, `CLAUDE.md`'de tek satır.

  ```
  # CLAUDE.md
  @AGENTS.md
  ```

  DOĞRULA İki araçta da aynı soruya aynı cevap.

2 Bağlam yönetimi

Bağlam doldukça model eski ayrıntıyı daha zayıf tutar, her mesaj daha pahalı olur ve sıkıştırma ayrıntı kaybettirir. En ucuz bağlam, hiç açılmamış olandır.

- [ ] **2.1 Bir özellik için bir oturum aç. İş bitince durumu dosyaya yaz, yeni oturuma geç.** TEMEL

  NEDEN Uzun oturum hem limit yer hem de sıkıştırmada kararları kaybettirir.

  NASIL Oturum sonunda Claude'a proje kökünde bir devir dosyası yazdır, yeni oturum oradan başlasın.

  ```
  Oturumu kapatıyoruz. BEKLEYENLER.md'ye yaz:
  1. Sıradaki adım (komut, dosya, beklenen sonuç)
  2. Açık sorular
  3. Alınan kararlar ve gerekçeleri, denenip olmayanlar
  4. Bitenler (tarihli, kısa)
  ```

  DOĞRULA Haftada kaç kez "özetle devam" gördüğünü say. Hedef sıfıra yakın.

- [ ] **2.2 Bağlamı tahminle değil sayıyla izle.** TEMEL

  NEDEN Doluluğu hissedemezsin. Sayı görünce devri zamanında yaparsın.

  NASIL `/context` ile anlık dağılım. `/statusline` ile alt satıra doluluk yüzdesi.

  ```
  /statusline model adını ve context_window.used_percentage değerini göster
  ```

  DOĞRULA Durum satırında yüzde görünüyor.

- [ ] **2.3 Konu değişince `/clear` de.** TEMEL

  NEDEN Eski konunun ayrıntısı yeni işte gürültüdür, yanlış varsayıma da yol açar.

  NASIL Alakasız yeni iş için yeni oturum ya da `/clear`.

  DOĞRULA Bir oturumda iki farklı projeye ait iş kalmamalı.

- [ ] **2.4 Çok dosya okuyan araştırmayı alt ajana ver.** İLERİ

  NEDEN Ham metin alt ajanın bağlamında kalır, ana oturuma yalnız sonuç döner.

  NASIL Dönüş biçimini baştan söyle.

  ```
  Bir alt ajanla src/ altındaki bütün ödeme akışını tara.
  Bana yalnız şunları dön: giriş noktaları (dosya:satır),
  harici servis çağrıları, hata yönetimi olmayan yerler.
  ```

  DOĞRULA Ana oturumda dosya dökümü yerine kısa rapor görürsün.

- [ ] **2.5 Büyük araç çıktısını dosyaya yazdır, bağlama yalnız özetini al.** İLERİ

  NEDEN Tek bir test logu on binlerce token yer.

  NASIL Çıktıyı dosyaya yönlendir, sonra ilgili satırları oku.

  ```
  npm test > /tmp/test.log 2>&1; tail -30 /tmp/test.log
  ```

  DOĞRULA `/context`'te araç çıktılarının payına bak.

- [ ] **2.6 `/compact` yaparken neyin korunacağını söyle.** İLERİ

  NEDEN Talimatsız sıkıştırma neyin önemli olduğunu tahmin eder.

  NASIL Komutun arkasına korunacakları yaz.

  ```
  /compact veri modeli kararlarını, açık hataları ve değişen dosya yollarını koru
  ```

  DOĞRULA Sıkıştırmadan sonra kritik bir kararı sor, doğru gelmeli.

3 Görev verme

Bir isteğin kalitesi, en çok neyin "bitti" sayılacağının açıklığına bağlı. Bitiş tanımı yoksa model ya erken durur ya da senden "devam" bekler.

- [ ] **3.1 Bitiş ölçütünü baştan yaz.** TEMEL

  NEDEN Ölçüt yoksa iş parça parça ilerler ve her parçada seni bekler.

  NASIL İsteğin sonuna bitişi ve tek durma noktasını ekle, ya da `/goal` ile koşul koy. Koşulu her turdan sonra küçük bir model değerlendirir ama dosya okumaz, yalnız konuşmadaki kanıta bakar. O yüzden kanıtın çıktıda görünmesini iste ve bir tur sınırı ekle.

  ```
  Planın 1–6 arası maddelerini sırayla bitir.
  Yalnız veritabanı şemasını değiştirmen gerekirse dur ve sor.
  Bitti sayılması için: npm test yeşil, yeni uç için 3 test var.

  /goal npm test çıktısı 0 hata gösteriyor ve yeni uç için 3 test var, ya da 20 tur oldu
  ```

  DOĞRULA Bir iş boyunca kaç kez "devam" yazdığını say.

- [ ] **3.2 Kabul ölçütünü doğrulanabilir yaz.** TEMEL

  NEDEN "Çalışsın" denetlenemez, "şu test 12/12 geçsin" denetlenir.

  NASIL Sayı, ekran ya da komut çıktısıyla tarif et.

  ```
  Kötü: Arama düzgün çalışsın.
  İyi:  "ist" yazınca 300 ms içinde İstanbul ilk sırada gelir;
        boş sorguda liste temizlenir; search.test.ts 8/8 geçer.
  ```

  DOĞRULA Ölçütü başka biri okuyup evet/hayır diyebiliyor mu?

- [ ] **3.3 Büyük ve belirsiz işte önce plan iste, küçük işte doğrudan yaptır.** TEMEL

  NEDEN Plan, yanlış yönü kod yazılmadan yakalar. Küçük işte plan zaman kaybıdır.

  NASIL Plan modu: CLI'da Shift+Tab ile modlar arasında geç ya da tek bir isteğin başına `/plan` yaz. Uygulamada mod seçiciyi kullan.

  DOĞRULA Planı onaylamadan dosya değişmemiş olmalı.

- [ ] **3.4 Kısıtları ve yapılmayacakları açıkça söyle.** TEMEL

  NEDEN Model boşluğu en olası çözümle doldurur. O çözüm çoğu zaman senin istemediğindir.

  NASIL İsteğe ayrı bir "Kısıtlar" bloğu ekle.

  ```
  Kısıtlar:
  - Yeni bağımlılık ekleme.
  - Sunucu tarafı yok, veri cihazda kalır.
  - Mevcut renk ve boşluk değerlerini kullan.
  ```

  DOĞRULA Çıktıda kısıtı ihlal eden bir öğe var mı?

- [ ] **3.5 İstediğin çıktının bir örneğini, mümkünse bir de karşı örneğini ver.** İLERİ

  NEDEN Bir örnek, bir paragraf tarifin anlatamadığı tonu ve biçimi taşır.

  NASIL "Şuna benzesin / şuna benzemesin" diye iki kısa örnek.

  ```
  Bildirim metni şuna benzesin: "Bugün 3 dakikan var mı? Bir sayfa kaldı."
  Şuna benzemesin: "Hedeflerinize ulaşmak için şimdi harekete geçin!"
  ```

  DOĞRULA İlk çıktı örneğe yakın mı?

- [ ] **3.6 Claude'dan soru sorarken önerdiği cevabı da yazmasını iste.** İLERİ

  NEDEN Boş soru işi sana geri yükler. Önerili soruya "önerinle devam" demek yeter.

  NASIL Global CLAUDE.md'ye tek kural.

  ```
  - Sorduğun her soruya önerdiğin cevabı ve gerekçesini ekle.
  - Seçenekleri harflendir, tek mesajda tek karar iste.
  ```

  DOĞRULA Gelen sorulara tek kelimeyle cevap verebiliyor musun?

- [ ] **3.7 Uzun bir brief'i yapıştırmadan önce eksiklerini buldur.** İLERİ

  NEDEN Brief'teki boşluk sonradan "beğenmedim" ve yeniden iş olarak döner.

  NASIL Ayrı bir oturumda eksik listesi iste, kendisi doldurmasın.

  ```
  Bu brief'i oku. Karar verilmemiş noktaları ve çelişkileri listele.
  Hiçbirini kendin doldurma, her biri için kime sorulacağını yaz.
  ```

  DOĞRULA Eksik listesi kapanmadan işi başlatma.

4 Prompt yazımı: önce ve sonra

İyi bir istek uzun olmak zorunda değil. Dört şey taşıması yeter: amaç, "iyi"nin tanımı, bağlam ve çıktının biçimi. Aşağıdaki örnekler aynı isteğin zayıf ve güçlü hâlleri.

- [ ] **4.1 "En iyi" ya da "kaliteli" diyorsan ölçütünü yaz.** TEMEL

  NEDEN Tanımsız sıfatı model kendi ortalamasıyla doldurur.

  NASIL Sıfatın yerine iki-üç gözlemlenebilir özellik koy.

  ```
  Önce: Bana en iyi onboarding ekranını tasarla.
  Sonra: İlk açılışta 30 saniyede bitecek, en fazla 3 ekranlık,
         izin isteğini ilk değer görüldükten sonra soran bir onboarding.
         Üç farklı yön çiz, her biri tam ekran.
  ```

  DOĞRULA Ölçütü okuyan biri çıktıyı puanlayabilir mi?

- [ ] **4.2 Okuru ve kullanım yerini söyle.** TEMEL

  NEDEN Aynı içerik yöneticiye, geliştiriciye ve müşteriye farklı yazılır.

  NASIL "Kim okuyacak, nerede, ne karar verecek" cümlesi.

  ```
  Önce: Bu sürüm için not yaz.
  Sonra: Bu sürüm notunu App Store'daki "Yenilikler" alanı için yaz.
         Okur teknik olmayan kullanıcı. En fazla 4 madde, her biri
         kullanıcının fark edeceği bir değişiklik.
  ```

  DOĞRULA Çıktının tonu ve ayrıntı düzeyi okura uyuyor mu?

- [ ] **4.3 Uzunluk istiyorsan dolguyu da yasakla.** İLERİ

  NEDEN "En az 10 sayfa" tek başına doldurma davetidir.

  NASIL Uzunluk hedefinin yanına "madde uydurma" kısıtı.

  ```
  En az 10 sayfa olsun ama sayfa sayısını tutturmak için madde uydurma.
  Gerekirse bölüm ekle. Genel tavsiye yerine somut davranış yaz.
  ```

  DOĞRULA Silinse kimsenin aramayacağı bir paragraf var mı?

- [ ] **4.4 Çıktının biçimini şablonla ver.** İLERİ

  NEDEN Biçim belirsizse her bölüm farklı yapıda gelir, karşılaştırılamaz.

  NASIL Her öğe için alan listesi.

  ```
  Her madde şu biçimde:
  - tek cümlelik eylem (fiille başlar)
  - neden önemli (bir cümle)
  - nasıl yapılır (komut, ayar ya da dosya)
  - nasıl doğrularım
  ```

  DOĞRULA Bütün öğeler aynı alanları taşıyor mu?

5 Model, effort ve maliyet

Her işe en güçlü modeli ve en yüksek düşünme bütçesini vermek limiti erken bitirir. Doğru eşleştirme, aynı limitle daha çok iş demek.

- [ ] **5.1 Alt ajanların varsayılan modelini Sonnet yap.** TEMEL

  NEDEN Okuma, tarama, doğrulama işlerinde en güçlü model gerekmez. Ana oturum yine güçlü modelde kalır.

  NASIL `~/.claude/settings.json` içinde tek satır. Opus isteyen ajan tanımına `model: opus` yazar.

  ```
  {
    "env": { "CLAUDE_CODE_SUBAGENT_MODEL": "sonnet" }
  }
  ```

  DOĞRULA Alt ajanlı bir koşumun `--output-format json` çıktısında iki ayrı model görünür.

- [ ] **5.2 Ana modeli takma adla seç, sürüme sabitleme.** TEMEL

  NEDEN Sabit sürüm, yeni model çıkınca sessizce eski kalır.

  NASIL `settings.json` 'da tam sürüm adı yerine takma ad.

  ```
  { "model": "opus" }
  ```

  DOĞRULA `claude -p ok --output-format json` çıktısında güncel model adı.

- [ ] **5.3 Düşünme bütçesini işe göre ayarla.** TEMEL

  NEDEN Mimari karar ile dosya yeniden adlandırma aynı bütçeyi istemez. "Think hard" gibi ifadeler sıradan metin olarak geçer, hiçbir ayarı değiştirmez. Yalnız `ultrathink` o tur için ek bir talimat ekler, effort ayarı yine aynı kalır.

  NASIL Oturumda `/effort`, başlatırken `--effort`. Kalıcı ayar `effortLevel`. Kullanılabilen seviyeler modele göre değişir.

  ```
  /effort high      # bu oturum için
  claude --effort max   # tek seferlik zor iş
  ```

  DOĞRULA Mimari işte high ya da max, mekanik işte low kullandığını gözle.

- [ ] **5.4 Kullanımı haftada bir kez sayıyla gör.** İLERİ

  NEDEN Limit bittiğinde değil, bitmeden önce neyin harcadığını bilmek gerekir.

  NASIL `/usage`.

  DOĞRULA Her hafta tek satır not: en çok harcayan proje.

- [ ] **5.5 Advisor'u zor kararlar için kullan.** İLERİ

  NEDEN Ana model karar noktalarında daha güçlü bir modele danışır. Her danışma, advisor modelinin ücretiyle ayrıca token harcar.

  NASIL `/advisor opus` ile aç, `/advisor off` ile kapat. Deneysel, yalnız Anthropic API ile çalışıyor.

  DOĞRULA `/advisor` ile bağlı modeli kontrol et.

6 İzinler

İzin ekranında bekleyen bir oturum iş üretmez. İzin listesi gözetimsiz koşumun ayakta kalmasını sağlar, deny listesi de onun güvenlik kemeri.

- [ ] **6.1 Her aktif projede `.claude/settings.json` izin listesi tut.** TEMEL

  NEDEN Gözetimsiz koşumu ayakta tutan en güvenilir şey projeye ait liste.

  NASIL Projenin derleme, test, commit komutlarını alt komut düzeyinde ekle.

  ```
  {
    "permissions": {
      "allow": [
        "Bash(npm test:*)",
        "Bash(npm run build:*)",
        "Bash(git add:*)",
        "Bash(git commit:*)"
      ]
    }
  }
  ```

  DOĞRULA Gözetimsiz bir koşumun log'unda izin bekleyen adım kalmamalı.

- [ ] **6.2 Yıkıcı komutları deny listesine koy.** TEMEL

  NEDEN Deny, allow'dan önce değerlendirilir. Geniş bir allow kuralı bile deny'ı aşamaz.

  NASIL Global ya da proje ayarında `deny` dizisi.

  ```
  "deny": [
    "Bash(rm -rf:*)",
    "Bash(git push --force:*)",
    "Bash(git reset --hard:*)"
  ]
  ```

  DOĞRULA Deny'daki komutu iste, reddedilmeli.

- [ ] **6.3 Allow girdisini dar tut.** İLERİ

  NEDEN Geniş bir joker, istemediğin alt komuta da izin verir.

  NASIL Araç adı yerine alt komut düzeyinde yaz.

  ```
  Geniş: "Bash(git:*)"            # push, reset, clean da dahil
  Dar:   "Bash(git status:*)", "Bash(git diff:*)", "Bash(git commit:*)"
  ```

  DOĞRULA Listede `:*` ile biten her girdinin en kötü kullanımını düşün.

- [ ] **6.4 Komutu izin motorunun eşleştirebileceği biçimde yaz.** İLERİ

  NEDEN `$(...)`, `${...}` ya da `source` içeren komut, allow listesinde olsa bile her seferinde sorabilir.

  NASIL venv'i aktive etme, yorumlayıcıyı doğrudan çağır.

  ```
  Soran:  source .venv/bin/activate && python -m pytest
  Sormayan: .venv/bin/python -m pytest
  ```

  DOĞRULA Allow'da olan bir komut yine soruyorsa içinde genişletme ara.

- [ ] **6.5 Tekrarlanan analizleri betiğe çevir.** İLERİ

  NEDEN Her seferinde yazılan anlık kod izin listesine giremez, her seferinde sorar.

  NASIL Analizi projede `scripts/` altına koy, betiği izin listesine ekle.

  DOĞRULA Aynı analiz ikinci kez istendiğinde betik çağrılıyor mu?

- [ ] **6.6 Geçmişten izin listesi önerisi al.** İLERİ

  NEDEN Hangi komutun sık sorulduğunu tahmin etmek yerine kayıttan çıkar.

  NASIL `/fewer-permission-prompts`.

  DOĞRULA Önerilen her girdiyi tek tek gözden geçir, yıkıcı olanı alma.

7 Güvenlik ve gizlilik

Claude dosyalarını okur, komut çalıştırır, web sayfası ve e-posta açar. Okuduğu her içerik bir talimat taşıyabilir ve yazdığı her şey bir yere gidebilir. Bu bölüm o iki yönü kapatır.

- [ ] **7.1 Gizli dosyaları okuma dışı bırak.** TEMEL

  NEDEN Anahtarın bağlama girmesi, sonra bir çıktıya ya da commit'e sızması demek.

  NASIL Deny listesine `Read` kuralları.

  ```
  "deny": [
    "Read(./.env)",
    "Read(./.env.*)",
    "Read(./**/*.pem)",
    "Read(./secrets/**)"
  ]
  ```

  DOĞRULA ".env dosyasını oku" iste, reddedilmeli.

- [ ] **7.2 Kurumsal veriyi kişisel hesaba ve depoya taşıma.** TEMEL

  NEDEN Şirket içi adresler, müşteri verisi ve iç doküman, özel bir depoda bile şirketin dışına çıkmış olur.

  NASIL Şirket işini şirketin onayladığı hesapla yap. Paylaşmadan önce iç alan adı ve iç kod deposu adresleri için tarama yaptır.

  ```
  Bu klasörü paylaşmadan önce tara: şirket alan adları, iç git adresleri,
  IP adresleri, e-posta adresleri, token'a benzeyen diziler.
  Bulduklarını dosya:satır olarak listele, hiçbirini silme.
  ```

  DOĞRULA Push öncesi tarama boş dönüyor.

- [ ] **7.3 Web sayfası, e-posta ve belgedeki talimata uyma refleksini kapat.** TEMEL

  NEDEN Okunan içerikteki "şunu yap" cümlesi senin talimatın değildir. Prompt injection böyle çalışır.

  NASIL Dış içerikle çalışırken izin modunu sıkı tut, yan etkili adımları (gönder, sil, öde) onaya bağla.

  DOĞRULA Dış içerikten gelen bir talimat Claude tarafından sana sorulmadan uygulanmamalı.

- [ ] **7.4 Web'den içerik çeken komutları onaya bağla.** İLERİ

  NEDEN Çekilen sayfa ya da dosya talimat taşıyabilir. `curl` ve `wget` varsayılan olarak otomatik onaylanmıyor, öyle kalsın.

  NASIL Bu komutları allow listesine ekleme. İstersen deny'a al. Güvenilmeyen içeriği doğrudan Claude'a pipe'lama.

  ```
  "deny": ["Bash(curl:*)", "Bash(wget:*)"]
  ```

  DOĞRULA İzin listende `curl` ya da `wget` yok.

- [ ] **7.5 Güvenmediğin kodda sandbox'ı aç.** İLERİ

  NEDEN İndirilen bir repodaki betik dosya sistemine ve ağa serbestçe erişmesin.

  NASIL `/sandbox` ya da `sandbox.enabled`.

  ```
  { "sandbox": { "enabled": true } }
  ```

  DOĞRULA Sandbox'ta proje dışına yazma denemesi reddedilmeli.

- [ ] **7.6 Tehlikeli komutu hook'la içeriğinden yakala.** İLERİ

  NEDEN Deny listesi kalıba bakar. Hook komutun tamamını okuyabilir.

  NASIL PreToolUse hook'u, çıkış kodu 2 çağrıyı engeller ve stderr modele gider.

```
#!/bin/sh
# PreToolUse, matcher: Bash
komut=$(jq -r '.tool_input.command')
case "$komut" in
  *"DROP TABLE"*|*"--force"*) echo "Engellendi: $komut" >&2; exit 2;;
esac
exit 0
```

  DOĞRULA Engellenecek komutu iste, hook'un mesajı görünmeli.

8 Hook'lar

Talimat bir rica, hook bir mekanizma. Her oturumda kesin olması gereken şey talimata değil hook'a yazılır.

- [ ] **8.1 Her oturuma girmesi gereken bilgiyi SessionStart hook'una koy.** İLERİ

  NEDEN Talimatı model unutabilir. Hook her oturumda çalışır.

  NASIL Betiğin stdout'u oturumun bağlamına girer.

```
"hooks": {
  "SessionStart": [{ "hooks": [{
    "type": "command",
    "command": "head -80 BEKLEYENLER.md 2>/dev/null || true"
  }]}]
}
```

  DOĞRULA Okuma araçları kapalı bir `claude -p` ile içerikten soru sor.

- [ ] **8.2 Biçimlendirmeyi düzenleme sonrası hook'a bağla.** İLERİ

  NEDEN Biçimi her seferinde istemek yerine her düzenlemeden sonra kendiliğinden düzelir.

  NASIL PostToolUse, düzenleme araçlarıyla eşleşen matcher.

```
"PostToolUse": [{
  "matcher": "Edit|Write",
  "hooks": [{ "type": "command", "command": "npx prettier --write ." }]
}]
```

  DOĞRULA Bozuk girintili bir dosya düzenlet, sonuç biçimli gelmeli.

- [ ] **8.3 Bağlam büyüyünce otomatik uyarı kur.** İLERİ

  NEDEN Uyarı devri sıkıştırmadan önce hatırlatır.

  NASIL UserPromptSubmit hook'u transcript'teki son yanıtın `usage` alanından token sayar, eşikte bir kez not düşer.

  DOĞRULA Eşiği çok düşük ayarlayıp notun düştüğünü gör, sonra gerçek değere çek.

- [ ] **8.4 Hook'u test et ve bozarak testin düştüğünü gör.** İLERİ

  NEDEN Hiçbir şeye bakmayan hook da sessizce "çalışır".

  NASIL Hook'a örnek bir JSON girdisi verip çıktısını kontrol eden küçük bir test.

```
echo '{"cwd":"'$PWD'"}' | sh .claude/hooks/oturum-basi.sh | head -3
```

  DOĞRULA Hook'u bilerek boz, test düşmeli.

- [ ] **8.5 Hook hata verirse oturumu durdurmasın.** İLERİ

  NEDEN Bir hook hatası bütün oturumları kilitleyebilir.

  NASIL Beklenmedik durumda sessizce 0 ile çık.

  DOĞRULA Hook'a bozuk girdi ver, oturum yine açılmalı.

9 Skill, komut, ajan, MCP

Aynı tarifi üçüncü kez yazıyorsan, o tarif bir dosyaya aittir. Skill gerektiğinde yüklenir, ajan kendi bağlamında çalışır, MCP dış sisteme API ile bağlanır.

- [ ] **9.1 Eleştiriyi yazandan ayrı bir bağlama ver.** TEMEL

  NEDEN Bir şeyi yazan model, onu gözden geçirirken onaylamaya meyillidir.

  NASIL Yazma aracı olmayan bir doğrulayıcı ajan ya da yeni oturum.

  ```
  ---
  name: dogrulayici
  description: Biten bir işin iddialarını bağımsız kontrol eder. "Bitti", "testler
  geçiyor" denince kullan.
  tools: Read, Grep, Glob, Bash
  model: sonnet
  ---
  Bu işi sen yapmadın. Onaylamak değil kırmak için bak.
  Her iddia için: kanıtı nerede, kanıt gerçekten bir şeye bakıyor mu?
  ```

  DOĞRULA Eleştiri en az bir somut bulgu içeriyor mu?

- [ ] **9.2 Üçüncü kez anlattığın işi skill'e çevir.** İLERİ

  NEDEN Tarif her seferinde biraz farklı anlatılır, sonuç da farklı çıkar.

  NASIL `~/.claude/skills/<ad>/SKILL.md`.

  ```
  ---
  name: surum-notu
  description: Son etiketten bu yana değişikliklerden kullanıcıya dönük sürüm notu
  yazar. "Sürüm notu", "release notes" denince kullan. Teknik changelog için kullanma.
  ---
  1. git log ile son etiketten bu yanaki commit'leri al.
  2. Kullanıcının fark edeceği değişiklikleri seç, en fazla 5.
  3. Her maddeyi kullanıcının diliyle bir cümlede yaz.
  ```

  DOĞRULA Yeni oturumda işi tarif etmeden iste, skill kendiliğinden yüklenmeli.

- [ ] **9.3 Skill açıklamasına "kullanma" sınırı koy.** İLERİ

  NEDEN Model skill'i açıklamadan seçer. Sınır yoksa komşu işlerde de yüklenir.

  NASIL Açıklamaya hem tetikleyici ifadeleri hem de kullanılmayacağı durumu yaz (yukarıdaki örnekteki son cümle).

  DOĞRULA Benzer ama farklı bir istekte skill yüklenmemeli.

- [ ] **9.4 Yan etkili skill'i kendiliğinden çalışmaya kapat.** İLERİ

  NEDEN Deploy ya da mesaj gönderen bir skill yanlış eşleşmede çalışmasın.

  NASIL Frontmatter'a tek satır, skill yalnız `/ad` ile çağrılır.

  ```
  disable-model-invocation: true
  ```

  DOĞRULA İlgili cümleyi yaz, skill kendiliğinden çalışmamalı.

- [ ] **9.5 Bir uygulamaya bağlanırken önce o uygulamanın MCP'sini dene.** İLERİ

  NEDEN API tabanlı araç hızlı ve kesin. Ekranda tıklayarak sürmek yavaş ve kırılgan.

  NASIL Sıra: uygulamanın MCP'si, sonra tarayıcı, en son bilgisayar kontrolü.

  DOĞRULA Aynı iş için kaç araç çağrısı gittiğine bak.

- [ ] **9.6 Kendi setini plugin olarak paketle.** İLERİ

  NEDEN Başka makineye ya da ekip arkadaşına tek komutla taşınır.

  NASIL `.claude-plugin/plugin.json` ve skill dizinleri.

  DOĞRULA Temiz bir makinede kurulumdan sonra skill listede görünmeli.

10 Ekiple kullanım

Bir kişinin iyi düzeni, ekip için de düzen olabilir. Ama ortak dosyaya giren her şey herkesin oturumunu etkiler.

- [ ] **10.1 Ortak ayarları `.claude/settings.json`'da, kişisel olanları `settings.local.json`'da tut.** İLERİ

  NEDEN Ekibin izin listesi ve hook'ları git ile paylaşılır, kişisel tercihin başkasına geçmez.

  NASIL `settings.json` commit'lenir, `settings.local.json` git dışında kalır. Öncelik sırası yukarıdan aşağı: yönetilen ayarlar, komut satırı, `settings.local.json`, `settings.json`, kullanıcı ayarları.

  DOĞRULA Yeni bir ekip üyesi repoyu açınca aynı izinlerle başlıyor.

- [ ] **10.2 Proje CLAUDE.md'sini kod gibi gözden geçir.** İLERİ

  NEDEN Bir kural bütün ekibin oturumunu değiştirir.

  NASIL CLAUDE.md değişikliği de pull request'ten geçsin.

  DOĞRULA Son ay eklenen kuralların gerekçesi commit mesajında yazıyor mu?

- [ ] **10.3 Ekipte tekrar eden işi ortak skill ya da plugin yap.** İLERİ

  NEDEN Herkes aynı işi kendi tarifinle yaptırınca sonuçlar tutarsız olur.

  NASIL Repoda `.claude/skills/` ya da ekip plugin'i.

  DOĞRULA İki kişinin aynı skill ile ürettiği çıktı aynı biçimde.

11 Paralel çalışma

Aynı anda çok oturum çalıştırmak güçlü bir kaldıraç. Ama her paylaşılan kaynak bir çakışma noktası ve makine yükü her şeyi yavaşlatır.

- [ ] **11.1 Aynı dosyalara iki yazar sokma. Her yazar ayrı worktree'de.** İLERİ

  NEDEN İki oturum aynı dosyayı düzenlerse biri diğerinin işini ezer.

  NASIL Worktree ile başlat.

  ```
  claude --worktree odeme-ekrani
  ```

  DOĞRULA `git worktree list`'te her yazar için ayrı satır.

- [ ] **11.2 Stage ile commit'i tek komutta yap.** İLERİ

  NEDEN Git index'i aynı repodaki oturumlar arasında ortak. Bekleyen stage başkasının commit'ine karışır.

  NASIL Dosyaları komutun sonunda say.

  ```
  git add src/odeme.ts && git commit -m "Ödeme: kart doğrulama" -- src/odeme.ts
  ```

  DOĞRULA Commit'te yalnız kendi dosyaların var.

- [ ] **11.3 Port ve tarayıcı oturumunu projeye özel yap.** İLERİ

  NEDEN Aynı porttaki iki sunucu birbirini düşürür, yanlış sayfa test edilir.

  NASIL Her projeye sabit ve farklı bir port.

  DOĞRULA Açılan sayfa gerçekten bu projenin sayfası.

- [ ] **11.4 Ağır oturum sayısını makine yüküne göre sınırla.** İLERİ

  NEDEN Yük yükseldikçe kısa komutlar zaman aşımına düşer, testler rastgele ölür.

  NASIL Başlatmadan önce `uptime`. Yük yüksekse yeni ağır iş açma, buluta gönder.

  DOĞRULA Gün içinde yük çekirdek sayısının birkaç katını geçmiyor.

- [ ] **11.5 Uzun gözetimsiz işi bulut oturumuna gönder.** İLERİ

  NEDEN Makineyi yormaz, uyku onu öldürmez.

  NASIL Uygulamada bulut hedefli oturum. Yerel cihaz isteyen iş yerelde kalır.

  DOĞRULA Bilgisayar kapalıyken iş ilerlemiş olmalı.

- [ ] **11.6 Çok ajanlı iş akışını yalnız gerçekten paralel işe kullan.** İLERİ

  NEDEN Onlarca ajan çok token harcar. Sıralı iş paralelden kazanmaz.

  NASIL Bağımsız parçalar için, örneğin on dosyanın ayrı ayrı incelenmesi.

  DOĞRULA İş akışı sonrası kullanım farkına `/usage`'dan bak.

12 Doğrulama

"Geçti" demek bir iddia, kanıt değil. En tehlikeli yeşil, hiçbir şeye bakmadığı için yeşil olandır.

- [ ] **12.1 Sonuçta durumu değil sayıyı oku.** TEMEL

  NEDEN Eşleşmeyen bir test filtresi sıfır test koşup başarılı diyebilir.

  NASIL Raporda test sayısını iste.

  ```
  Yeşil görünen ama boş:  Test run with 0 tests ... SUCCEEDED
  Gerçek:                 Executed 42 tests, with 0 failures
  ```

  DOĞRULA Her "geçti"nin yanında bir sayı.

- [ ] **12.2 Yeni kontrolü hatayı geri koyarak doğrula.** TEMEL

  NEDEN Kontrol gerçekten bakıyorsa bozuk kodda düşmeli.

  NASIL Kodu bilerek boz, testi koş, düştüğünü gör, geri al.

  ```
  Yazdığın testin gerçekten çalıştığını kanıtla: fiyat hesabındaki
  indirimi geçici olarak kaldır, testi koş, düştüğünü göster, sonra geri al.
  ```

  DOĞRULA Bozuk sürümde kırmızı.

- [ ] **12.3 Alt ajanın raporunu kanıtla karşılaştır.** TEMEL

  NEDEN Ajan raporu bir iddiadır.

  NASIL Diff'e, dosyaya, çıktıya bak. Ondan sonra "bitti" de.

  DOĞRULA Rapordaki bir iddiayı rastgele seç ve doğrula.

- [ ] **12.4 İş sonunda üç satır istet: kaçırılan, en zayıf iddia, gerekçesiz karar.** İLERİ

  NEDEN Övgü değil kırma denemesi, sorunun kendisini sana getirir.

  NASIL CLAUDE.md'ye tek kural ya da bir slash komutu.

  ```
  İş bitince sormadan üç satır yaz:
  1. Neyi kaçırmış olabilirim (bakmadığım dosya, koşmadığım yol)
  2. Hangi iddiam en zayıf
  3. Hangi kararım gerekçesiz
  ```

  DOĞRULA Üç satırın her biri somut bir dosya ya da karar adı içeriyor mu?

- [ ] **12.5 Görsel değişikliği önce/sonra ekran görüntüsüyle gör.** İLERİ

  NEDEN "Ne değişti" sorusu metinle cevaplanmaz.

  NASIL İki ekran görüntüsü yan yana.

  DOĞRULA Farkı ilk bakışta görebiliyor musun?

- [ ] **12.6 Önemli değişikliği teslimden önce kod incelemesinden geçir.** İLERİ

  NEDEN Yazan bağlam kendi hatasını kaçırır.

  NASIL `/code-review`.

  DOĞRULA İnceleme en az bir bulgu ya da açık bir "yok" dönmeli.

13 Gözetimsiz koşum

"Ben yatıyorum, sabaha bitir" dediğin iş, sabah çalışan bir sonuç bırakmalı. Gece boyu izin ekranında beklemek ya da sessizce ölmek en pahalı başarısızlık.

- [ ] **13.1 Gözetimsiz koşum için talimat setini önceden yaz.** İLERİ

  NEDEN "Sormadan ilerle" demek, hangi durumda ne yapılacağını söylemez.

  NASIL Global CLAUDE.md'ye bir bölüm.

  ```
  ## Ben yokken
  - Soru sorma, onay bekleme. Kararı ver, varsayımı yaz.
  - İzin gerektiren adımı en sona bırak.
  - Bir maddede tıkanırsan atla, kalanı bitir.
  - Sonda yaz: yapılan, yapılmayan ve neden, varsayımlar.
  ```

  DOĞRULA Bir koşumu dene, sabah rapor eksiksiz mi?

- [ ] **13.2 Ayrılmadan önce dört şeye bak: izin listesi, disk, yük, uyku.** İLERİ

  NEDEN Bu dördünden biri geceyi boşa çıkarır.

  NASIL Tek komutla dördünü gör.

  ```
  cat .claude/settings.json | head -20; df -h .; uptime; pmset -g | grep sleep
  ```

  DOĞRULA Dördü de yeşil.

- [ ] **13.3 Uzun işi kaldığı yerden devam edecek biçimde kur.** İLERİ

  NEDEN Uyku ya da çökme işi baştan başlatmasın.

  NASIL Ara çıktıyı diske yaz, her adım öncesi "bu adım bitti mi" kontrolü.

  DOĞRULA İşi yarıda öldür, yeniden başlat, tamamlanan adımlar atlanmalı.

- [ ] **13.4 Zamanlanmış oturumda uzun işi arka plana atıp turu kapatma.** İLERİ

  NEDEN Arka plan süreci oturumla birlikte ölebilir ve geriye log bile kalmayabilir.

  NASIL Turu açık tut, log'u oturumdan bağımsız bir yola yaz.

  DOĞRULA Sabah log'un son satırı işin bitişini gösteriyor mu?

- [ ] **13.5 İş bitince açık kaynakları kapat.** İLERİ

  NEDEN Açık kalan simülatör, sunucu ya da tarayıcı belleği ve diski tüketir.

  NASIL Koşum sonunda kendi açtığın süreçleri kapat, başka oturumunkine dokunma.

  DOĞRULA Sabah yalnız beklediğin süreçler açık.

14 Rutinler ve zamanlama

Tekrarlanan işi elle tetiklemek unutulur. Ama bilgisayar kapanınca ölen bir rutin de güvenilir değildir.

- [ ] **14.1 Yerel dosya istemeyen rutini bulut rutinine taşı.** İLERİ

  NEDEN Masaüstü uygulamasındaki zamanlanmış görev yalnız uygulama açık ve bilgisayar uyanıkken koşar. Kaçan koşum 7 gün içinde yalnız bir kez telafi edilir.

  NASIL `/schedule` (claude.ai girişi gerekir, araştırma önizlemesi).

  DOĞRULA Bilgisayar kapalı bir günde rutinin çıktısı gelmiş mi?

- [ ] **14.2 Headless rutinde çıktıyı ve tur sayısını sınırla.** İLERİ

  NEDEN Sınırsız koşum takılı kalınca saatlerce harcar.

  NASIL JSON çıktı, tur sınırı, kendi log yolu.

  ```
  claude -p "$(cat gorev.md)" --output-format json --max-turns 30 \
    >> ~/loglar/bulten-$(date +%F).json
  ```

  DOĞRULA Log'da her koşumun maliyet satırı var.

- [ ] **14.3 Rutinin sessiz ölümünü ayrı bir nöbetle yakala.** İLERİ

  NEDEN Hiç koşmayan rutin hata da vermez.

  NASIL Son başarılı koşumun zamanını kontrol eden ikinci küçük bir rutin.

  DOĞRULA Bir rutini bilerek durdur, nöbet uyarmalı.

- [ ] **14.4 Dış site tarayan rutinde istek saatlerini dağıt.** İLERİ

  NEDEN Her gün aynı saatte gelen istek engellenmeye davetiye.

  NASIL Rastgele ön bekleme, gün içinde farklı saatler.

  DOĞRULA Log'daki istek saatleri farklı mı?

15 Git ve yayın

Git, Claude ile çalışırken en güvenilir geri alma düğmesi. Sık commit, Claude'a cesur değişiklik yaptırmanın güvencesi.

- [ ] **15.1 Her projeyi git altına al ve commit'i Claude'a bırak.** TEMEL

  NEDEN Küçük commit küçük geri alma demek. `.bak` kopyaları geri alma değil, karmaşa üretir.

  NASIL `git init`, sonra CLAUDE.md'ye commit kuralı.

  ```
  - Sormadan commit at, mantıklı parçalara böl.
  - Push ve deploy yalnız açıkça istendiğinde.
  ```

  DOĞRULA `git log --oneline`'da her satır tek bir değişikliği anlatıyor.

- [ ] **15.2 Checkpoint'in sınırını bil.** TEMEL

  NEDEN `/rewind` yalnız dosya araçlarıyla yapılan düzenlemeleri geri alır. Bash'le silinen ya da taşınanı almaz.

  NASIL Bash'le toplu değişiklikten önce commit.

  DOĞRULA Riskli toplu işten önce `git status` temiz mi?

- [ ] **15.3 Push ve deploy'u izin listesine koyma.** İLERİ

  NEDEN Uzak depoya giden şey geri alınması zor bir paylaşımdır.

  NASIL `git push` allow'da olmasın, her seferinde sorsun.

  DOĞRULA İzin listelerinde push yok.

- [ ] **15.4 Commit atfını talimatla değil ayarla yönet.** İLERİ

  NEDEN Talimat unutulabilir, ayar her commit'te uygulanır.

  NASIL `attribution` ayarı (`includeCoAuthoredBy` eskidi). Boş metin o atfı gizler, `false` hepsini gizler (v2.1.281 ve sonrası).

  ```
  { "attribution": { "commit": "", "pr": "" } }
  ```

  DOĞRULA Ayardan sonraki ilk commit'te atıf satırı beklediğin gibi.

16 Tasarım ve arayüz

Tasarımda en pahalı döngü "beğenmedim, tekrar dene". Onu kısaltan şey araştırma, birden çok yön ve görsel karşılaştırma.

- [ ] **16.1 Tasarımdan önce gerçek örneklerden ilham topla.** İLERİ

  NEDEN İlhamsız ilk taslak şablona yakın çıkar.

  NASIL Kaynaklı bir ilham raporu iste.

  ```
  Alışkanlık uygulamalarından 8 örnek bul, ana ekranlarını karşılaştır.
  Her biri için: neyi iyi yapıyor, bizim uygulamaya ne alınabilir, kaynak.
  ```

  DOĞRULA Her öneri bir gerçek örneğe dayanıyor mu?

- [ ] **16.2 İki-üç farklı yönü tam ekran çizdir.** İLERİ

  NEDEN Anlatılan tasarım kafada farklı canlanır, alt alanlar unutulur.

  NASIL Her yönü alt alanlar dahil tam ekran iste.

  DOĞRULA Seçimini görsele bakarak yapabiliyor musun?

- [ ] **16.3 Beğenmediğinde aynı yönü değil, farklı bir yönü iste.** İLERİ

  NEDEN Rötuş aynı fikrin varyasyonunu üretir.

  NASIL Neyin çalışmadığını söyle ve yön değişikliği iste.

  ```
  Bu yönden çık. Kart dizisi yerine tek bir odak alanı dene,
  renk değil boşluk ve tipografiyle hiyerarşi kur.
  ```

  DOĞRULA Yeni öneri ilkinden yapısal olarak farklı mı?

- [ ] **16.4 Tasarım dilini bir kez yaz, her turda yeniden anlatma.** İLERİ

  NEDEN Renk, tipografi ve boşluk kuralları her seferinde sohbetle aktarılınca kayar.

  NASIL Projede tasarım ilkeleri dosyası ya da tasarım skill'i.

  DOĞRULA Yeni oturumda tasarım dili sorulunca dosyadan cevap gelmeli.

17 Mobil ve simülatör

iOS ya da Android uygulaması geliştiriyorsan simülatör en çok kullanılan araç olur. Paylaşılan cihaz, elle tıklama ve açık kalan simülatör en sık zaman kaybı.

- [ ] **17.1 Her projeye kendi simülatörünü aç.** İLERİ

  NEDEN Paylaşılan simülatörde başka oturumun uygulaması öne gelir, senin uygulaman bozuk sanılır.

  NASIL Proje adıyla bir cihaz oluştur, CLAUDE.md'ye adını yaz.

  ```
  xcrun simctl create "FiyatIzleyici" "iPhone 16 Pro"
  ```

  DOĞRULA Ekran görüntüsünde senin uygulaman var.

- [ ] **17.2 Tekrarlanan arayüz akışını otomatik teste çevir.** İLERİ

  NEDEN Elle tıklayarak doğrulama her seferinde aynı maliyeti öder.

  NASIL Maestro, XCUITest ya da Espresso akışı, tek komutla koşan.

  ```
  appId: com.ornek.fiyat
  ---
  - launchApp
  - tapOn: "Ürün ekle"
  - inputText: "Kahve"
  - assertVisible: "Kahve"
  ```

  DOĞRULA Akış bozuk bir ekranda kırmızı dönmeli.

- [ ] **17.3 Simülatöre Türkçe karakterli metin yazdırma.** İLERİ

  NEDEN Otomasyon aracı ASCII dışı karakteri sessizce düşürebilir, otomatik düzeltme isimleri değiştirebilir.

  NASIL Testte ASCII isim kullan, Türkçe metni koddan ya da birim testten doğrula.

  DOĞRULA Girilen metin ekranda birebir görünüyor.

- [ ] **17.4 Farklı ekran boyutlarında kontrol et.** İLERİ

  NEDEN Küçük ekranda taşan metin büyük ekranda görünmez.

  NASIL En küçük ve en büyük desteklenen cihazda ekran görüntüsü.

  DOĞRULA İki görüntüde de taşma yok.

- [ ] **17.5 İş bitince kendi simülatörünü kapat ve sıfırla.** İLERİ

  NEDEN Açık simülatörler belleği ve diski tüketir. Dolan disk testi hata vermeden asar.

  NASIL Yalnız projenin kendi cihazı, adıyla bulunan UDID.

  ```
  xcrun simctl shutdown <UDID> && xcrun simctl erase <UDID>
  ```

  DOĞRULA `simctl list`'te projenin cihazı Shutdown.

18 Bilgi birikimi

Öğrenilen ders yazılmazsa bir sonraki oturumda yeniden öğrenilir. Yanlış yere yazılırsa da kaybolur.

- [ ] **18.1 Dersi doğru katmana yaz.** İLERİ

  NEDEN Makine gerçeği bir proje dosyasında kalırsa diğer projelere görünmez.

  NASIL "Bu cümle nerede doğru olmaktan çıkıyor" diye sor.

  ```
  Makine ve araç gerçeği   -> ~/.claude/CLAUDE.md
  Koda ait gerçek          -> proje CLAUDE.md
  Repo + bu makine         -> CLAUDE.local.md
  Tarihli ölçüm, kanıt     -> ayrı kanıt dosyası
  ```

  DOĞRULA Her yeni kural doğru dosyada mı?

- [ ] **18.2 "Sonra" ve "unutma" denen her şeyi hemen dosyaya yazdır.** İLERİ

  NEDEN Oturum içi hafıza devirde kaybolur.

  NASIL Proje kökünde bekleyenler dosyası, CLAUDE.md'de buna dair tek kural.

  DOĞRULA "Neler kaldı" sorusu dosyadan cevaplanıyor.

- [ ] **18.3 Aynı düzeltme üçüncü kez geldiyse kural yap.** İLERİ

  NEDEN Üçüncü tekrar artık tesadüf değil.

  NASIL Düzeltmeyi say, üçte kurala çevir.

  DOĞRULA Son ayın düzeltmelerinden kaçı kurala dönüştü?

- [ ] **18.4 Kuralı kısa, kanıtını ayrı tut.** İLERİ

  NEDEN Kural her mesajda okunur, kanıtın gövdesi yalnız gerektiğinde.

  NASIL Kuralın sonunda kanıt dosyasına bir atıf.

  DOĞRULA Her global kuralın bir kanıt atfı var mı?

19 Bakım ritmi

Ayarlar, kurallar ve izin listeleri zamanla eskir. Küçük ve düzenli bakım, büyük temizlikten ucuzdur.

- [ ] **19.1 Haftalık: kullanımı ve bekleyenleri gözden geçir.** İLERİ

  NEDEN Limiti ve yarım işleri hafta bitmeden görürsün.

  NASIL `/usage` ve aktif projelerin bekleyenler dosyası.

  DOĞRULA Haftalık not: en çok harcayan proje, en eski bekleyen madde.

- [ ] **19.2 Haftalık: eski worktree'leri ve diski temizle.** İLERİ

  NEDEN Dolan disk testleri hata vermeden asabilir.

  NASIL Kullanılmayan worktree'leri kaldır, boş alana bak.

  ```
  git worktree list; git worktree prune; df -h .
  ```

  DOĞRULA Boş alan yüzde 15'in üstünde.

- [ ] **19.3 Aylık: CLAUDE.md'leri buda.** İLERİ

  NEDEN Kurallar birikir, bazıları artık hiç tetiklenmez.

  NASIL Her kural için: son ay işe yaradı mı? Yaramadıysa sil ya da skill'e taşı.

  DOĞRULA Dosya boyutu aydan aya büyümüyor.

- [ ] **19.4 Aylık: izin listelerini ve hook testlerini gözden geçir.** İLERİ

  NEDEN Yeni komutlar eklenir, Claude Code güncellemesi hook girdisini değiştirebilir.

  NASIL `/fewer-permission-prompts`, ardından hook testleri.

  DOĞRULA Gözetimsiz koşumlarda izin bekleyen adım yok, testler yeşil.

- [ ] **19.5 Çeyrekte bir: kendi kullanım analizini çıkar.** İLERİ

  NEDEN Alışkanlığın değişip değişmediğini sayı söyler.

  NASIL Claude'dan oturum kayıtlarını taramasını iste.

  ```
  ~/.claude/projects altındaki oturum kayıtlarını tara. Çıkar:
  mesaj sayısı, en sık kısa mesajlar ("devam" gibi), bağlam dolma sayısı,
  alt ajan çağrılarının model dağılımı. Önceki çeyrekle karşılaştır.
  ```

  DOĞRULA Sonuçları önceki çeyrekle karşılaştır.

Kendi puan kartın

Her bölüm için kaç maddenin yapıldığını, kaçının sırada olduğunu yaz. Üç ay sonra yeniden doldur.

#	BÖLÜM	MADDE	YAPILDI	SIRADA	GEREK YOK
1	Talimat katmanları	6			
2	Bağlam yönetimi	6			
3	Görev verme	7			
4	Prompt yazımı: önce ve sonra	4			
5	Model, effort ve maliyet	5			
6	İzinler	6			
7	Güvenlik ve gizlilik	6			
8	Hook'lar	5			
9	Skill, komut, ajan, MCP	6			
10	Ekiple kullanım	3			
11	Paralel çalışma	6			
12	Doğrulama	6			
13	Gözetimsiz koşum	5			
14	Rutinler ve zamanlama	4			
15	Git ve yayın	4			
16	Tasarım ve arayüz	4			
17	Mobil ve simülatör	5			
18	Bilgi birikimi	4			
19	Bakım ritmi	5			
	Toplam	**97**			

Claude Code 2.1.284 · Ekim 2026. Özellikler hızlı değişiyor; bir komut ya da ayar adı çalışmıyorsa code.claude.com/docs adresindeki güncel belgeye bak.